```
# Please fill in your student number here for checkoff:
8025320082
```

```
8025320082
```

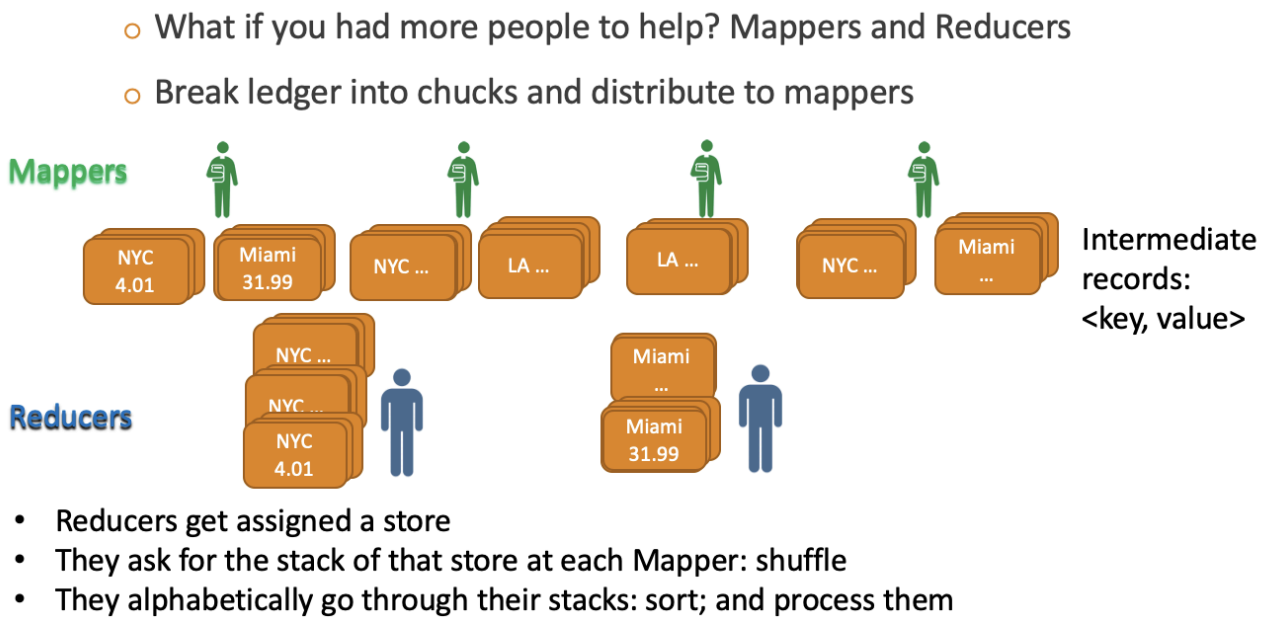
Hadoop streaming emulator

--Prof. Dorien Herremans

You can run this lab in Playground mode (File - Open in playground mode) or make a copy. After you filled in and run all the cells submit this as an .ipynb ifile on eDimension.

This lab serves to practice MapReduce logic. Instead of communicating with a real Hadoop cluster, we are using an emulator for Hadoop streaming. This will allow us to use the exact same commands and see the result, but without the hassle of installing a virtual machine...

Before we begin, here is a figure that recaps MapReduce's function.



Let's download the emulator from github:

```
!git clone https://github.com/hideki1234/pyhdemu.git
```

```
Cloning into 'pyhdemu'...
remote: Enumerating objects: 41, done.
remote: Total 41 (delta 0), reused 0 (delta 0), pack-reused 41 (from 1)
Receiving objects: 100% (41/41), 19.66 KiB | 447.00 KiB/s, done.
Resolving deltas: 100% (4/4), done.
```

Run hadoop streaming emulator (hdemu.py). This demo mapper/reducers counts the occurrence of words of the files in the pyhdemu/sample/input/ folder.

Let's first explore this folder. Remember, we can execute unix commands (just like if we were using a terminal, by starting the command with "!".

Which files are in the folder "pyhdemu/sample/input/"?

```
!ls pyhdemu/sample/input/
```

```
file1.txt  file2.txt
```

There are two text files. Let's have a look at the first few lines of file1.txt:

```
!head pyhdemu/sample/input/file1.txt
```

```
one two three
two three four
three four five
four five six
four five six seven
```

```
five six seven
five six seven eight
six seven eight
six seven eight seven
seven eight eight
```

There is a demo reducer and mapper in the folder as well. Let's explore them. We can display the content of a file using the 'cat' command.

```
!cat ./pyhdemu/sample/wc_mapper.py
```

```
#!/usr/bin/env python

from __future__ import print_function

import sys
import re

def main():
    pattern = re.compile("[A-Za-z][0-9A-Za-z]*")

    for line in sys.stdin:
        for word in pattern.findall(line):
            print('{}\t{}'.format(word.lower(), 1))

if __name__ == '__main__':
    main()
```

What does this mapper code do? It takes as input each line of the input files, puts the words in lower case, and outputs pairs:

theword [tab] 1

And let's have a look at the reducer code.

```
!cat ./pyhdemu/sample/wc_reducer.py
```

```
#!/usr/bin/env python

from __future__ import print_function

import sys

def emit(word, count):
    print('{}\t{}'.format(word, count))

def main():
    last_word = None
    last_count = 0
    for line in sys.stdin:
        word, count = line.split('\t', 1)
        try:
            count = int(count)
        except ValueError:
            continue

        if last_word == word:
            last_count += count
        else:
            if last_word:
                emit(last_word, last_count)
            last_word = word
            last_count = count

    if last_word:
        emit(last_word, last_count)


if __name__ == '__main__':
    main()
```

The reducer code takes an input all of the word pairs, line by line. Note that, as always in Hadoop, the reducer input is **sorted**. That allows us to just loop through the input and assume that once the word is different then the previous word, we can finalize the count of the previous word.

The output of the reducer is a list of pairs:

(thisword, count)

Now let's run the mapper / reducer on the input directory. This will process all the files in the input folder and save the output in the sample/output folder.

Note, the only difference with running this on a real Hadoop cluster is that instead of the emulator `hdemu.py`, you would call `hd` (hadoop).

```
!python ./pyhdemu/hdemu.py -input ./pyhdemu/sample/input -output ./pyhdemu/sample/output -mapper ./pyhdemu/sample/wc_mapper
```

```
System     : 3.12.13 (main, Mar  4 2026, 09:23:07) [GCC 11.4.0]
Mapper     : /content/pyhdemu/sample/wc_mapper.py
Reducer    : /content/pyhdemu/sample/wc_reducer.py
Input path : /content/pyhdemu/sample/input
Output path: /content/pyhdemu/sample/output
interim dir: None
**** mapping ****
**** shuffling ****
**** reducing ****
**** mapreduce job completed ****
```

If you get an error:

`!!!! ERROR !!!! Output path '/content/pyhdemu/sample/output' already exists`

That means you have already run the command! Even if your script has an error during running, each run needs to have a unique output folder. It will save the logs and output results in that folder. So if you get this error, just change the name of your output folder.

Now what was the result of this mapreduce task? Let's check the output folder.

```
!ls pyhdemu/sample/output/
```

```
part-00000  _SUCCESS
```

The file `part-0000` will contain our output. If the output is very long, it will be split in multiple files.

```
!cat pyhdemu/sample/output/part-00000
```

```
eight   8
finally 1
first   1
five    5
four    4
is      4
last    1
line    4
one     1
second  1
seven   7
six     6
the     4
third   1
this    4
three   3
two     2
```

Great! Now let's explore the example from in class...

Class example - counting sales per store

We will go over the first example together. We have a file: purchases.txt which contains the individual sales per store. Our goal is to get a list of total purchases per store.

First we will download the purchases files. They are available at https://dorienherremans.com/drop/CDS/hadoop/purchases.txt.gz

```
!wget https://dorienherremans.com/drop/CDS/hadoop/purchases.txt.gz
```

```
--2026-05-03 14:09:31--  https://dorienherremans.com/drop/CDS/hadoop/purchases.txt.gz
Resolving dorienherremans.com (dorienherremans.com)... 184.154.70.198
Connecting to dorienherremans.com (dorienherremans.com)|184.154.70.198|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 38454568 (37M) [application/x-gzip]
Saving to: ‘purchases.txt.gz’

purchases.txt.gz    100%[===================>]  36.67M  20.6MB/s    in 1.8s

2026-05-03 14:09:33 (20.6 MB/s) - ‘purchases.txt.gz’ saved [38454568/38454568]
```

We can create a data folder and move the uncompressed file in there:

```
!mkdir data  # make a folder called data
!gunzip purchases.txt.gz # unzip the file
!mv purchases.txt data/ # move it into the data folder
```

Let's have a look at how our data file looks like. Using the `head -n` command in unix, we can show the first 10 lines of this huge file.

```
!head -10 data/purchases.txt
```

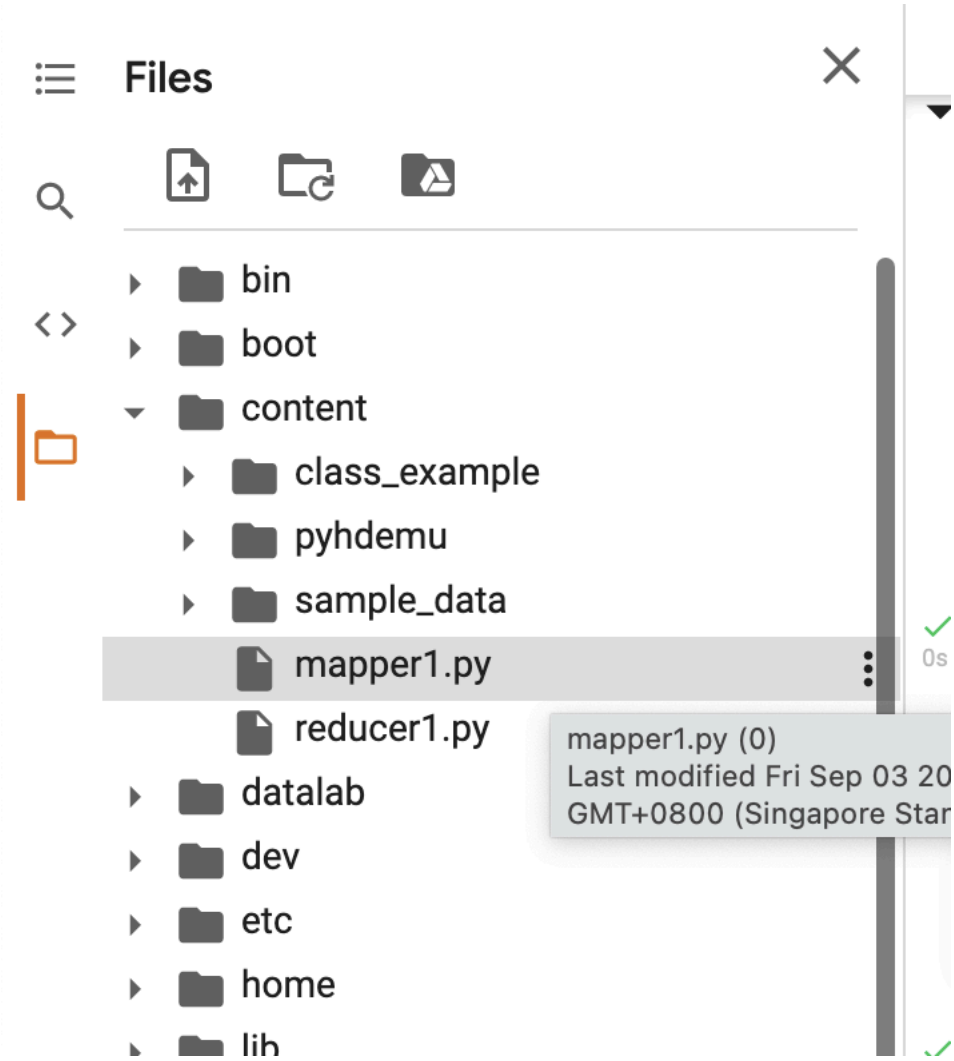
```
2012-01-01	09:00	San Jose	Men's Clothing	214.05	Amex
2012-01-01	09:00	Fort Worth	Women's Clothing	153.57	Visa
2012-01-01	09:00	San Diego	Music	66.08	Cash
2012-01-01	09:00	Pittsburgh	Pet Supplies	493.51	Discover
2012-01-01	09:00	Omaha	Children's Clothing	235.63	MasterCard
2012-01-01	09:00	Stockton	Men's Clothing	247.18	MasterCard
2012-01-01	09:00	Austin	Cameras	379.6	Visa
2012-01-01	09:00	New York	Consumer Electronics	296.8	Cash
2012-01-01	09:00	Corpus Christi	Toys	25.38	Discover
2012-01-01	09:00	Fort Worth	Toys	213.88	Visa
```

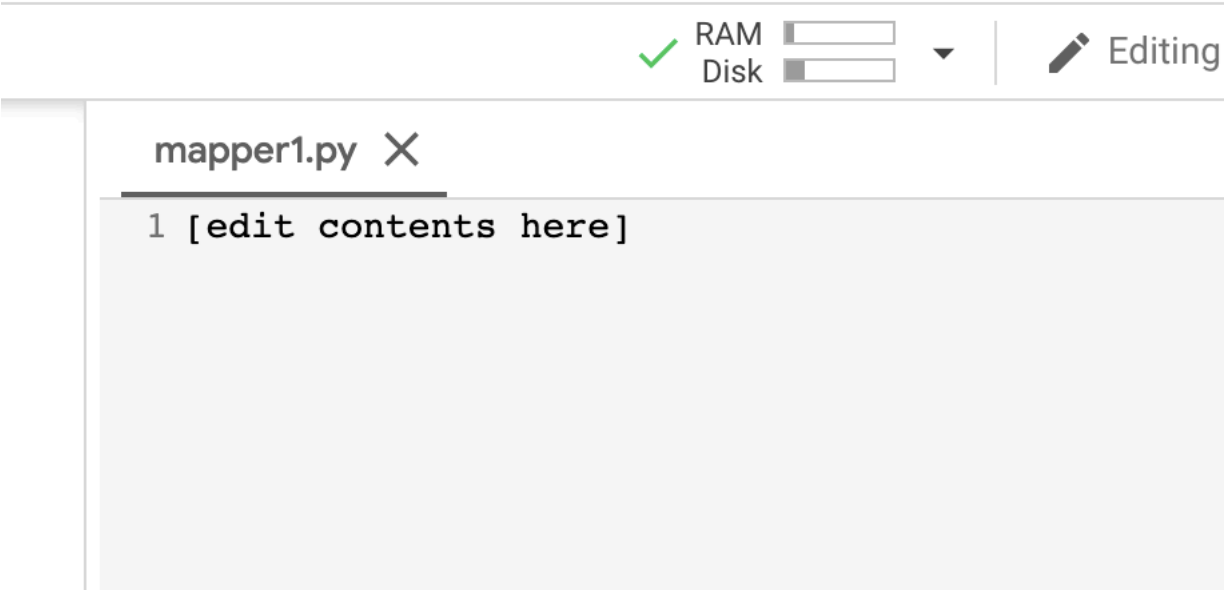
Now lets create our mapper and reducer files. You can use the `Files -> upload` command on the left to upload your python files or create them directly on Colab.

I'll start by creating a mapper and reducer file.

```
!touch "mapper1.py"
!touch "reducer1.py"
```

Now you can find the file in the file menu on the left, and edit it on the right.





So what to put in the files? As per the class example, we are using the following. **You have to copy this into the code below into the mapper1.py and reducer1.py files**. This will return the total purchases per store.

The mappers will go through all the input and create 'pairs' -> store with sales total of that transaction.

`mapper1.py`

```
#!/usr/bin/env python3

# Format of each line is:
# date\ttime\tstore name\titem description\tcost\tmethod of payment
#
# We want elements 2 (store name) and 4 (cost)
# We need to write them out to standard output, separated by a tab

%%writefile mapper1.py
#!/usr/bin/env python3
import sys

for line in sys.stdin:
    data = line.strip().split("\t")

    if len(data) == 6:
        date, time, store, item, cost, payment = data
        print("{0}\t{1}".format(store, cost))
```

```
Overwriting mapper1.py
```

These pairs are then sorted, and fed to the reducers. These reducers loop through the pairs and output a total for each store.

`reducer1.py`

```
%%writefile reducer1.py
#!/usr/bin/env python3
import sys

salesTotal = 0
oldKey = None

for line in sys.stdin:
    data_mapped = line.strip().split("\t")

    if len(data_mapped) != 2:
        continue

    thisKey, thisSale = data_mapped

    if oldKey and oldKey != thisKey:
        print(oldKey, "\t", salesTotal)
        salesTotal = 0
```

```
        oldKey = thisKey
        salesTotal += float(thisSale)

    if oldKey:
        print(oldKey, "\t", salesTotal)
```

```
Overwriting reducer1.py
```

```
!head -n 1 mapper1.py
!head -n 1 reducer1.py
```

```
#!/usr/bin/env python3
#!/usr/bin/env python3
```

```
!rm -rf output_2
```

3. Now you are ready to run the `hdemu.py` command. Make sure you point to the right directories.

```
!python ./pyhdemu/hdemu.py -input ./data -output output_2 -mapper mapper1.py -reducer reducer1.py
```

```
System     : 3.12.13 (main, Mar  4 2026, 09:23:07) [GCC 11.4.0]
Mapper     : /content/mapper1.py
Reducer    : /content/reducer1.py
Input path : /content/data
Output path: /content/output_2
interim dir: None
**** mapping ****
**** shuffling ****
**** reducing ****
**** mapreduce job completed ****
```

Was the output successful? Let's check the output folder.

```
!ls output_2
```

```
part-00000  _SUCCESS
```

The success flag is there, that's a good sign :) Now let's have a look at the output 'part-0000'

```
!head output_2/part-00000
```

```
Albuquerque      10052311.42000002
Anaheim          10076416.359999916
Anchorage        9933500.400000054
Arlington        10072207.969999947
Atlanta          9997146.700000068
Aurora   9992970.92000003
Austin   10057158.89999997
Bakersfield      10031208.919999944
Baltimore        10096521.450000003
Baton Rouge      10131273.230000038
```

Now this is a really long file, if we are looking for a particular city, we can use the grep command:

```
!grep Spokane output_2/part-00000
```

```
Spokane          10083362.979999933
```

Voila. It's as easy as that. Now this gets particularly powerful when we are dealing with immense unstructured data files. Let's explore more exercises...

Lab exercises:

2. Instead of breaking the sales down by store, instead retrieve a **sales breakdown by product category** across all of our stores (instead of by city). Create new mapper and reducer files for this. Hint, only 1 word needs to change.

```
#mapper
%%writefile mapper1.py
#!/usr/bin/env python3
import sys

for line in sys.stdin:
    data = line.strip().split("\t")
```

```
    if len(data) == 6:
        date, time, store, item, cost, payment = data


        print("{0}\t{1}".format(item, cost))
```

```
Overwriting mapper1.py
```

```
#reducer
#!/usr/bin/env python3
import sys

salesTotal = 0
oldKey = None

for line in sys.stdin:
    data_mapped = line.strip().split("\t")

    if len(data_mapped) != 2:
        continue

    thisKey, thisSale = data_mapped

    if oldKey and oldKey != thisKey:
        print(oldKey, "\t", salesTotal)
        salesTotal = 0

    oldKey = thisKey
    salesTotal += float(thisSale)

if oldKey:
    print(oldKey, "\t", salesTotal)
```

```
!rm -rf output_2
!python ./pyhdemu/hdemu.py -input ./data -output output_2 -mapper mapper1.py -reducer reducer1.py
!head output_2/part-00000
```

```
System     : 3.12.13 (main, Mar  4 2026, 09:23:07) [GCC 11.4.0]
Mapper     : /content/mapper1.py
Reducer    : /content/reducer1.py
Input path : /content/data
Output path: /content/output_2
interim dir: None
**** mapping ****
**** shuffling ****
**** reducing ****
**** mapreduce job completed ****
Baby     57491808.43999965
Books    57450757.91000004
CDs      57410753.04000111
Cameras          57299046.64000087
Children's Clothing      57624820.94000126
Computers        57315406.319999866
Consumer Electronics     57452374.12999909
Crafts   57418154.50000017
DVDs     57649212.13999929
Garden   57539833.109999545
```

link text3. Find the monetary value for the highest individual sale for each separate store.

```
#mapper
%%writefile mapper3.py
#!/usr/bin/env python3
import sys

for line in sys.stdin:
    data = line.strip().split("\t")

    if len(data) == 6:
        date, time, store, item, cost, payment = data
        print("{0}\t{1}".format(store, cost))
```

```
Writing mapper3.py
```

```
%%writefile reducer3.py
#!/usr/bin/env python3
import sys

current_store = None
max_sale = 0
```

```
for line in sys.stdin:
    data = line.strip().split("\t")

    if len(data) != 2:
        continue

    store, cost = data
    cost = float(cost)

    if current_store == store:
        if cost > max_sale:
            max_sale = cost
    else:
        if current_store:
            print(current_store, "\t", max_sale)
        current_store = store
        max_sale = cost

if current_store:
    print(current_store, "\t", max_sale)
```

```
Writing reducer3.py
```

```
!rm -rf output_3
!python ./pyhdemu/hdemu.py -input ./data -output output_3 -mapper mapper3.py -reducer reducer3.py
!head output_3/part-00000
```

```
System     : 3.12.13 (main, Mar  4 2026, 09:23:07) [GCC 11.4.0]
Mapper     : /content/mapper3.py
Reducer    : /content/reducer3.py
Input path : /content/data
Output path: /content/output_3
interim dir: None
**** mapping ****
**** shuffling ****
**** reducing ****
**** mapreduce job completed ****
Albuquerque      499.98
Anaheim          499.98
Anchorage        499.99
Arlington        499.95
Atlanta          499.96
Aurora   499.97
Austin   499.97
Bakersfield      499.97
Baltimore        499.99
Baton Rouge      499.98
```

4.Find the total sales value across all the stores, and the total number of sales. Assume there is only one reducer.

```
%%writefile mapper4.py
#!/usr/bin/env python3
import sys

for line in sys.stdin:
    data = line.strip().split("\t")

    if len(data) == 6:
        cost = data[4]
        print("ALL\t{0}".format(cost))
```

```
Writing mapper4.py
```

```
%%writefile reducer4.py
#!/usr/bin/env python3
import sys

total = 0
count = 0

for line in sys.stdin:
    data = line.strip().split("\t")

    if len(data) != 2:
        continue

    key, cost = data
    total += float(cost)
    count += 1
```

```
print("Total Sales:\t", total)
print("Number of Sales:\t", count)
```

```
Writing reducer4.py
```

```
!rm -rf output_4
!python ./pyhdemu/hdemu.py -input ./data -output output_4 -mapper mapper4.py -reducer reducer4.py
!cat output_4/part-00000
```

```
System     : 3.12.13 (main, Mar  4 2026, 09:23:07) [GCC 11.4.0]
Mapper     : /content/mapper4.py
Reducer    : /content/reducer4.py
Input path : /content/data
Output path: /content/output_4
interim dir: None
**** mapping ****
**** shuffling ****
**** reducing ****
**** mapreduce job completed ****
Total Sales:     1034457953.2600112
Number of Sales:         4138476
```

Answer key

You can check if you did it correctly through the following answers:

```
2. Total sales for Toys = 57,463,477.11
3. Highest individual sale for Reno = 499.99
4. Number of sales = 4,138,476, with total value = 1,034,457,953.26
```

```
  File "/tmp/ipykernel_3325/2048720485.py", line 3
    4. Number of sales = 4,138,476, with total value = 1,034,457,953.26
                                                     ^
SyntaxError: leading zeros in decimal integer literals are not permitted; use an 0o prefix for octal integers
```

Next steps: Explain error

link text## Advanced exercises

Common log file: use access_log as input. You can find the file here: https://dorienherremans.com/drop/CDS/hadoop/access_log.gz

`10.223.157.186 - - [15/Jul/2009:15:50:35 -0700] "GET /assets/js/lowpro.js HTTP/1.1" 200 10469 %h %l %u %t \"%r\" %>s %b`

```
%h is the IP address of the client
%l is identity of the client, or "-" if it's unavailable
%u is username of the client, or "-" if it's unavailable
%t is the time that the server finished processing the request. The format is [day/month/year:hour:minute:second zone]
%r is the request line from the client is given (in double quotes). It contains the method, path, query-string, and protocol o
%>s is the status code that the server sends back to the client. You will see see mostly status codes 200 (OK - The request ha
%b is the size of the object returned to the client, in bytes. It will be "-" in case of status code 304.
You'll have to write new Mappers and Reducers
```

5. Write a MapReduce program to find number of hits for each different file on the Web site.

```
!mkdir log_data
!mv access_log log_data/
```

```
mv: cannot stat 'access_log': No such file or directory
```

```
!head log_data/access_log
```

```
head: cannot open 'log_data/access_log' for reading: No such file or directory
```

```
%%writefile mapper_hits.py
#!/usr/bin/env python3
import sys

for line in sys.stdin:
    parts = line.strip().split()

    if len(parts) > 6:
```

```
        file_requested = parts[6]   # ✅ FIXED INDEX
        print(f"{file_requested}\t1")
```

```
Writing mapper_hits.py
```

```
%%writefile reducer_hits.py
#!/usr/bin/env python3
import sys

current_file = None
count = 0

for line in sys.stdin:
    file, value = line.strip().split("\t")
    value = int(value)

    if current_file == file:
        count += value
    else:
        if current_file:
            print(f"{current_file}\t{count}")
        current_file = file
        count = value

if current_file:
    print(f"{current_file}\t{count}")
```

```
Writing reducer_hits.py
```

```
!rm -rf output_hits
!python ./pyhdemu/hdemu.py -input ./log_data -output output_hits -mapper mapper_hits.py -reducer reducer_hits.py
!head output_hits/part-00000
```

```
System     : 3.12.13 (main, Mar  4 2026, 09:23:07) [GCC 11.4.0]
Mapper     : /content/mapper_hits.py
Reducer    : /content/reducer_hits.py
Input path : /content/log_data
Output path: /content/output_hits
interim dir: None
**** mapping ****
**** shuffling ****
**** reducing ****
**** mapreduce job completed ****
```

```
!cat log_data/access_log | python mapper_hits.py | grep "/assets/js/the-associates.js" | wc -l
```

```
cat: log_data/access_log: No such file or directory
0
```

6. Write a MapReduce program which determines the number of hits to the site made by each different IP address.

```
%%writefile mapper_ip.py
#!/usr/bin/env python3
import sys

for line in sys.stdin:
    parts = line.strip().split()

    if len(parts) > 0:
        ip = parts[0]   # ✅ IP is first element
        print(f"{ip}\t1")
```

```
Writing mapper_ip.py
```

```
%%writefile reducer_ip.py
#!/usr/bin/env python3
import sys

current_ip = None
count = 0

for line in sys.stdin:
    ip, value = line.strip().split("\t")
    value = int(value)

    if current_ip == ip:
        count += value
    else:
        if current_ip:
            print(f"{current_ip}\t{count}")
```

```
        current_ip = ip
        count = value

if current_ip:
    print(f"{current_ip}\t{count}")
```

```
Writing reducer_ip.py
```

```
!rm -rf output_ip
!python ./pyhdemu/hdemu.py -input ./log_data -output output_ip -mapper mapper_ip.py -reducer reducer_ip.py
```

```
System     : 3.12.13 (main, Mar  4 2026, 09:23:07) [GCC 11.4.0]
Mapper     : /content/mapper_ip.py
Reducer    : /content/reducer_ip.py
Input path : /content/log_data
Output path: /content/output_ip
interim dir: None
**** mapping ****
**** shuffling ****
**** reducing ****
**** mapreduce job completed ****
```

```
!head output_ip/part-00000
```

```
!cat log_data/access_log | awk '{print $1}' | grep "10.99.99.186" | wc -l
```

```
cat: log_data/access_log: No such file or directory
0
```

7. Find the most popular file on the website, i.e. whose file path occurs most often in access_log. Your reducer should output the file's path and number of times it appears in the log

```
%%writefile mapper_popular.py
#!/usr/bin/env python3
import sys

for line in sys.stdin:
    parts = line.strip().split()

    if len(parts) > 5:
        file_requested = parts[5]
        print(f"{file_requested}\t1")
```

```
Writing mapper_popular.py
```

```
%%writefile reducer_popular.py
#!/usr/bin/env python3
import sys

current_file = None
count = 0

max_file = None
max_count = 0

for line in sys.stdin:
    file, value = line.strip().split("\t")
    value = int(value)

    if current_file == file:
        count += value
    else:
        if current_file:
            # check for max
            if count > max_count:
                max_count = count
                max_file = current_file

        current_file = file
        count = value

# check last file
if current_file:
    if count > max_count:
        max_count = count
        max_file = current_file

# final output
print(f"{max_file}\t{max_count}")
```

```
Writing reducer_popular.py
```

```
!rm -rf output_popular
!python ./pyhdemu/hdemu.py -input ./log_data -output output_popular -mapper mapper_popular.py -reducer reducer_popular.py
```

```
System     : 3.12.13 (main, Mar  4 2026, 09:23:07) [GCC 11.4.0]
Mapper     : /content/mapper_popular.py
Reducer    : /content/reducer_popular.py
Input path : /content/log_data
Output path: /content/output_popular
interim dir: None
**** mapping ****
**** shuffling ****
**** reducing ****
**** mapreduce job completed ****
```

```
!cat log_data/access_log | python mapper_hits.py | sort | python reducer_hits.py | sort -k2 -nr | head -1
```

```
cat: log_data/access_log: No such file or directory
```

Quick check for answers:

- Hit for the page /assets/js/the-associates.js = 2456
- Hits made by IP address 10.99.99.186 = 6
- Most popular file's pathname = combined.css
- Number of occurrences = 117348

```
import requests
import re

# download text
url = "http://www.gutenberg.org/cache/epub/11/pg11.txt"
text = requests.get(url).text

# count exact word only
count = len(re.findall(r"\bCheshire\b", text))

print("Cheshire count:", count)
```

```
Cheshire count: 7
```

```
import requests

url = "https://www.gutenberg.org/files/100/100-0.txt"
response = requests.get(url)
response.raise_for_status()

with open("shakespeare.txt", "w", encoding="utf-8") as f:
    f.write(response.text)

# Question 3
with open("shakespeare.txt", encoding="utf-8") as f:
    words = f.read().split()
lengths = [len(word) for word in words]
lengths.sort()
n = len(lengths)

if n % 2 == 0:
    median = (lengths[n//2 - 1] + lengths[n//2]) / 2
else:
    median = lengths[n//2]
print("Median word length:", median)
```

```
Median word length: 4.0
```